

Словари

В прошлых уроках мы изучили четыре типа коллекций в Python:

- списки – изменяемые коллекции элементов, индексируемые;
- строки – неизменяемые коллекции символов, индексируемые;
- кортежи – неизменяемые коллекции элементов, индексируемые;
- множества – изменяемые коллекции уникальных элементов, неиндексируемые.

Следующий тип – **словари** – изменяемые коллекции элементов с **произвольными индексами – ключами**. Если в списках элементы индексируются целыми числами, начиная с 0, то в словарях – любыми ключами, в том числе в виде строк.

Как нам уже известно, списки – удобный и самый популярный способ хранения большого количества данных в одной переменной. Списки индексируют все хранящиеся в них элементы, что позволяет быстро обращаться к элементу, зная его индекс.

Приведенный ниже код:

```
languages = ['Python', 'C#', 'Java', 'C++']  
  
print(languages[0])  
print(languages[2])
```

ВЫВОДИТ:

```
Python
```

```
Java
```

Допустим, мы хотим хранить имя создателя каждого языка программирования. Это можно сделать несколькими способами.

Способ 1. Хранить еще один список, где по соответствующему индексу будет находиться имя создателя языка программирования.

Приведенный ниже код:

```
languages = ['Python', 'C#', 'Java', 'C++']
creators = ['Гвидо ван Россум', 'Андерс Хейлсберг', 'Джеймс Гослинг', 'Бьёрн Страуструп']

print('Создателем языка', languages[0], 'является', creators[0])
```

ВЫВОДИТ:

Создателем языка Python является Гвидо ван Россум

Подход рабочий, но хранить данные в двух коллекциях не очень удобно.

Способ 2. Хранить список кортежей с парами значений "язык - имя создателя" в каждом.

Приведенный ниже код:

```
languages = [('Python', 'Гвидо ван Россум'),
              ('C#', 'Андерс Хейлсберг'),
              ('Java', 'Джеймс Гослинг'),
              ('C++', 'Бьёрн Страуструп')]

print('Создателем языка', languages[2][0], 'является',
      languages[2][1])
```

ВЫВОДИТ:

Создателем языка Java является Джеймс Гослинг

Тоже рабочий подход, однако не очень эффективный. Придется написать цикл `for` для поиска по всем элементам списка `languages` кортежа, первый элемент которого равен искомому (названию языка). Чтобы найти автора языка C++ , нужно будет в цикле пройти мимо Python, C# и Java. Не получится угадать заранее, что язык C++ лежит после них.

Приведенный ниже код:

```
languages = [('Python', 'Гвидо ван Россум'),
              ('C#', 'Андерс Хейлсберг'),
              ('Java', 'Джеймс Гослинг'),
              ('C++', 'Бьёрн Страуструп')]

for item in languages:
    if item[0] == 'C++':
        print('Создателем языка', item[0], 'является', item[1])
```

ВЫВОДИТ:

Создателем языка C++ является Бьёрн Страуструп

Списки индексируются целыми числами, но в этом случае удобно было бы находить информацию не по числу, а по строке — названию языка программирования. В списках строки не могут быть индексами, однако в словарях это возможно.

Словарь (тип данных `dict`), как и список, позволяет хранить много данных. В отличие от списка, в словаре для каждого элемента можно произвольно определить «индекс» — **ключ**, по которому он будет доступен.

Словарь — реализация структуры данных "ассоциативный массив" или "хеш-таблица". В других языках аналогичная структура называется `map`, `HashMap`, `Dictionary`.

Создание словаря

Чтобы создать словарь, нужно перечислить его элементы — пары ключ-значение — через запятую в фигурных скобках, как и элементы множества. Первым указывается ключ, после двоеточия — значение, доступное в словаре по этому ключу.

Приведенный ниже код:

```
languages = {'Python': 'Гвидо ван Россум',
             'C#': 'Андерс Хейлсберг',
             'Java': 'Джеймс Гослинг',
             'C++': 'Бьёрн Страуструп'}
```

создает словарь, в котором ключом служит строка — название языка программирования, а значением — имя создателя языка.

Обращение к элементу словаря

Извлечь значение элемента словаря можно, обратившись к нему по его ключу. Чтобы получить значение по заданному ключу, как и в списках, используем квадратные скобки `[]`, индексируем по ключу.

Способ 3. Приведенный ниже код:

```
languages = {'Python': 'Гвидо ван Россум',
             'C#': 'Андерс Хейлсберг',
             'Java': 'Джеймс Гослинг',
```

```
'C++': 'Бьёрн Страуструп'}
```

```
print('Создателем языка C# является', languages['C#'])
```

ВЫВОДИТ:

```
Создателем языка C# является Андерс Хейлсберг
```

В отличие от списков, номеров позиций в словарях нет.

Приведенный ниже код:

```
languages = {'Python': 'Гвидо ван Россум',  
             'C#': 'Андерс Хейлсберг',  
             'Java': 'Джеймс Гослинг',  
             'C++': 'Бьёрн Страуструп'}
```

```
print('Создателем языка C# является', languages[1])
```

приводит к возникновению ошибки `KeyError`.

Ошибка `KeyError` возникнет и при попытке извлечь значение по несуществующему ключу. В качестве ключа можно указать выражение: Python вычислит его значение и обратится к искомому элементу.

Способ 4. Приведенный ниже код:

```
languages = {'Python': 'Гвидо ван Россум',  
             'C#': 'Андерс Хейлсберг',  
             'Java': 'Джеймс Гослинг',  
             'C++': 'Бьёрн Страуструп'}
```

```
print('Создателем языка C# является', languages['C' + '#'])
```

ВЫВОДИТ:

```
Создателем языка C# является Андерс Хейлсберг
```

Создание словаря с помощью функции `dict()`

Если ключи словаря — строки без каких-либо специальных символов, то для создания словаря можно использовать функцию `dict()`.

Приведенный ниже код:

```
info = dict(name = 'Timur', age = 28, job = 'Teacher')
```

создает словарь с тремя элементами, ключами которого служат строки 'name', 'age', 'job', а значениями — 'Timur', 28, 'Teacher'.

Создание словаря на основании списков и кортежей

Создавать словари можно на основе списков кортежей или кортежей списков. Первый элемент списка или кортежа станет ключом, второй — значением.

Приведенный ниже код:

```
info_list = [('name', 'Timur'), ('age', 28), ('job', 'Teacher')]
# список кортежей
```

```
info_dict = dict(info_list) # создаем словарь на основе списка
кортежей
```

создает словарь с тремя элементами, где ключи — строки name, age, job, а соответствующие им значения — 'Timur', 28, 'Teacher'.

Аналогично работает приведенный ниже код:

```
info_tuple = ('name', 'Timur'), ('age', 28), ('job', 'Teacher')
# кортеж списков
```

```
info_dict = dict(info_tuple) # создаем словарь на основе кортежа
списков
```

Если необходимо создать словарь, каждому ключу которого соответствует одно и то же значение, можно воспользоваться методом `fromkeys()`.

Приведенный ниже код:

```
dict1 = dict.fromkeys(['name', 'age', 'job'], 'Missed
information')
```

создает словарь с тремя элементами, где ключи — строки 'name', 'age', 'job', а соответствующие им значения: 'Missed information', 'Missed information', 'Missed information'.

Если методу `fromkeys()` не передать второй параметр, то по умолчанию присваивается значение `None`.

Приведенный ниже код:

```
dict1 = dict.fromkeys(['name', 'age', 'job'])
```

создает словарь с тремя элементами, в которых ключи — строки 'name', 'age', 'job', а значения — None, None, None.

Пустой словарь

Пустой словарь можно создать двумя способами:

- с помощью пустых фигурных скобок;
- с помощью функции `dict()`.

Приведенный ниже код:

```
dict1 = {}  
dict2 = dict()
```

```
print(dict1)  
print(dict2)  
print(type(dict1))
```

```
print(type(dict2))
```

создает два пустых словаря и выводит:

```
{}  
{}  
<class 'dict'>  
  
<class 'dict'>
```

Вспомните, что создать пустое множество можно, только используя функцию `set()`, потому что пустые фигурные скобки зарезервированы для создания словаря.

Вывод словаря

Для вывода всего словаря можно использовать функцию `print()`:

```
languages = {'Python': 'Гвидо ван Россум',  
             'C#': 'Андерс Хейлсберг',  
             'Java': 'Джеймс Гослинг'}
```

```
info = dict(name = 'Timur', age = 28, job = 'Teacher')
```

```
print(languages)
```

```
print(info)
```

Функция `print()` выводит на экран элементы словаря в фигурных скобках, разделенные запятыми:

```
{'Python': 'Гвидо ван Россум', 'C#': 'Андерс Хейлсберг', 'Java':  
'Джеймс Гослинг'}
```

```
{'name': 'Timur', 'age': 28, 'job': 'Teacher'}
```

Начиная с версии Python 3.6, словари являются упорядоченными, то есть сохраняют порядок следования ключей в порядке их внесения в словарь.

Примечания

Примечание 1. Словари принципиально отличаются от списков по структуре хранения в памяти. Список — последовательная область памяти, то есть все его элементы (указатели на элементы) действительно хранятся в указанном порядке, расположены последовательно. Благодаря этому и можно быстро «прыгнуть» к элементу по его индексу. В словаре же используется специальная структура данных — **хеш-таблица**. Она позволяет вычислять числовой хеш от ключа и использовать обычные списки, где в качестве индекса элемента берется этот хеш.

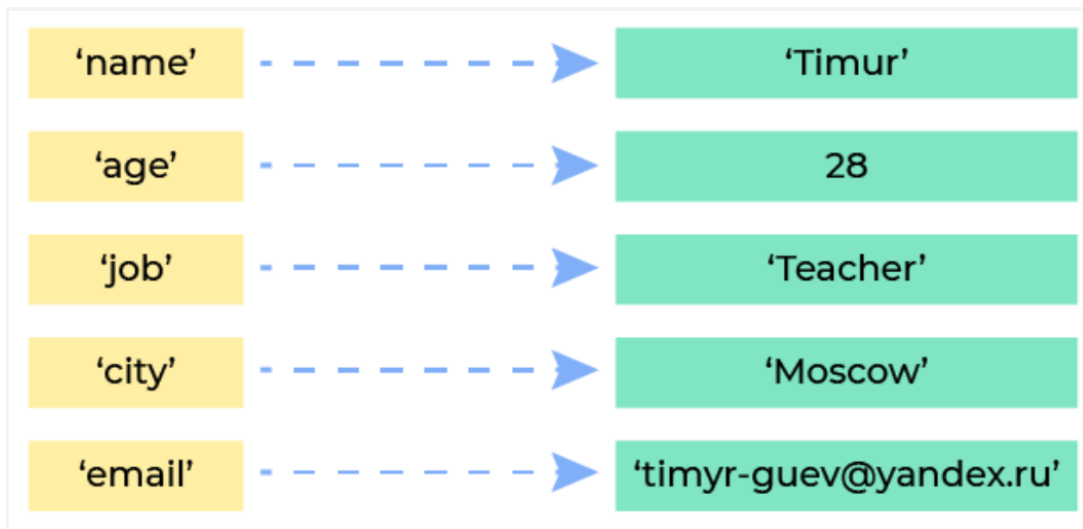
Примечание 2. В рамках одного словаря каждый ключ уникален.

Примечание 3. Словари удобно использовать для хранения различных сущностей. Например, если нужно работать с информацией о человеке, то можно хранить все необходимые сведения, включающие такие разные сущности как "возраст", "профессия", "название города", "адрес электронной почты" в одном словаре `info` и легко обращаться к его элементам по ключам:

```
info = {'name': 'Timur',  
        'age': 28,  
        'job': 'Teacher',  
        'city': 'Moscow',  
        'email': 'timyr-guev@yandex.ru'}
```

```
print(info['name'])
```

```
print(info['email'])
```



Примечание 4. Создать словарь на основании двух списков (кортежей) можно с помощью встроенной функции-упаковщика `zip()`, о которой расскажем позже.

Приведенный ниже код:

```
keys = ['name', 'age', 'job']
values = ['Timur', 28, 'Teacher']

info = dict(zip(keys, values))

print(info)
```

выводит (порядок элементов может отличаться):

```
{'name': 'Timur', 'age': 28, 'job': 'Teacher'}
```

В случае несовпадения длины списков функция самостоятельно отсечет лишние элементы.

Особенности словарей

Словари (тип данных `dict`) довольно просты, но о нескольких моментах следует помнить при их использовании.

Ключи должны быть уникальными

Словарь не может иметь два и более значений по одному и тому же ключу. Если при создании словаря (в литеральной форме) указать дважды один и тот же ключ, будет использовано последнее из указанных значений.

Приведенный ниже код:

```
info = {'name': 'Ruslan',  
        'age': 28,  
        'name': 'Timur'}
```

```
print(info['name'])
```

ВЫВОДИТ:

```
Timur
```

Ключи должны быть неизменяемым типом данных

Ключом словаря могут быть данные любого неизменяемого типа:

- число;
- строка;
- булево значение;
- кортеж;
- замороженное множество (frozenset);
- ...

Приведенный ниже код создает словарь, ключами которого являются неизменяемые типы данных:

```
my_dict = {198: 'beegeek', 'name': 'Bob', True: 'a', (2, 2): 25}
```

Ключ словаря не может относиться к изменяемому типу данных:

- список;
- множество;
- словарь;
- ...

Приведенный ниже код приводит к возникновению ошибки:

```
my_dict = {[2, 2]: 25, {1, 2}: 'python', 'name': 'Bob'}
```

Значения могут относиться к любому типу данных, их тип данных произволен

Нет никаких ограничений для значений, хранящихся в словарях. Значения в словарях могут принадлежать к произвольному типу данных и повторяться для разных ключей многократно.

```
my_dict1 = {'a': [1, 2, 3], 'b': {1, 2, 3}} # значения –  
изменяемый тип данных
```

```
my_dict2 = {'a': [1, 2], 'b': [1, 2], 'c': [1, 2]} # значения  
повторяются
```

Основы работы со словарями

Работа со словарями похожа на работу со списками, поскольку и словари, и списки содержат в качестве отдельных элементов пары: в словарях **ключ: значение**, в списках **индекс: значение**.

Функция len()

Длиной словаря называется количество его элементов. Для определения длины словаря используют встроенную функцию `len()` (от слова length – длина).

Следующий программный код:

```
fruits = {'Apple': 70, 'Grape': 100, 'Banana': 80}  
capitals = {'Россия': 'Москва', 'Франция': 'Париж'}  
  
print(len(fruits))  
  
print(len(capitals))
```

выведет:

3

2

Оператор принадлежности in

Оператор `in` позволяет проверить, содержит ли словарь заданный **ключ**.

Рассмотрим код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}
```

```
if 'Франция' in capitals:
```

```
    print('Столица Франции - это', capitals['Франция'])
```

Такой код проверяет, содержит ли словарь `capitals` элемент с ключом `Франция` и выводит соответствующий текст:

```
Столица Франции - это Париж
```

Можно использовать оператор `in` вместе с логическим оператором `not`.

Не забывайте, что при обращении по **несуществующему ключу**, возникнет ошибка во время выполнения программы.

Приведенный ниже код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}
```

```
print(capitals['Англия'])
```

приводит к возникновению ошибки:

```
KeyError: 'Англия'
```

Оператор принадлежности `in` на словарях работает **очень быстро**, намного быстрее, чем на списках, поэтому если нужен многократный поиск в коллекции данных, словарь – подходящий выбор.

Встроенные функции `sum()`, `min()`, `max()`

Встроенная функция `sum()` принимает в качестве аргумента **словарь с числовыми ключами** и вычисляет сумму его ключей.

Следующий программный код:

```
my_dict = {10: 'Россия', 20: 'США', 30: 'Франция'}
```

```
print('Сумма всех ключей словаря =', sum(my_dict))
```

ВЫВОДИТ:

```
Сумма всех ключей словаря = 60
```

Для корректной работы функции `sum()` ключами словаря должны быть именно числа.

Встроенные функции `min()` и `max()` принимают в качестве аргумента словарь и находят минимальный и максимальный ключ соответственно, при этом ключ может принадлежать к любому типу данных, для которого возможны операции порядка `<`, `<=`, `>`, `>=` (числа, строки, и т.д.).

Следующий программный код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}
months = {1: 'Январь', 2: 'Февраль', 3: 'Март'}

print('Минимальный ключ =', min(capitals))

print('Максимальный ключ =', max(months))
```

ВЫВОДИТ:

Минимальный ключ = Россия

Максимальный ключ = 3

Сравнение словарей

Словари можно сравнивать между собой. Равные словари имеют одинаковое количество элементов и содержат равные элементы (ключ: значение). Для сравнения словарей используются операторы `==` и `!=`.

Приведенный ниже код:

```
months1 = {1: 'Январь', 2: 'Февраль'}
months2 = {1: 'Январь', 2: 'Февраль', 3: 'Март'}
months3 = {3: 'Март', 1: 'Январь', 2: 'Февраль'}

print(months1 == months2)
print(months2 == months3)

print(months1 != months3)
```

ВЫВОДИТ:

False

True

True

Примечания

Примечание 1. Обращение по индексу и срезы **недоступны** для словарей.

Примечание 2. Операция конкатенации `+` и умножения на число `*` **недоступны** для словарей.

Примечание 3. Словари нужно использовать в следующих случаях:

- Подсчет числа каких-то объектов. В этом случае нужно завести словарь, в котором ключи — названия объектов, а значения — их количество.
- Хранение каких-либо данных, связанных с объектом. Ключи — наименования объектов, значения — связанные с ними данные.
Например, если нужно по названию месяца определить его порядковый номер, то это можно сделать при помощи словаря `num = {'January': 1, 'February': 2, 'March': 3, ...}`.
- Установка соответствия между объектами (например, “родитель—потомок”). Ключ — объект, значение — соответствующий ему объект.
- Если нужен обычный список, где максимальное значение индекса элемента очень велико, но при этом используются не все возможные индексы (так называемый “разреженный список”), то для экономии памяти можно использовать словарь.

Перебор элементов словаря

Перебор элементов словаря осуществляется так же как перебор элементов списка, с помощью цикла `for`.

Для вывода ключей словаря **каждого на отдельной строке** можно использовать следующий код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}
```

```
for key in capitals:
```

```
    print(key)
```

Такой код выведет (порядок элементов может отличаться):

```
Россия  
Франция  
  
Чехия
```

Для вывода значений словаря **каждого на отдельной строке** можно использовать следующий код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия':  
'Прага'}  
  
for key in capitals:  
  
    print(capitals[key])
```

Такой код выведет (порядок элементов может отличаться):

```
Москва  
Париж  
  
Прага
```

Для вывода элементов словаря **каждого на отдельной строке** можно использовать следующий код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия':  
'Прага'}  
  
for key in capitals:  
  
    print('Столица', key, '- это', capitals[key])
```

Такой код выведет (порядок элементов может отличаться):

```
Столица Россия - это Москва  
Столица Франция - это Париж  
  
Столица Чехия - это Прага
```

Методы keys(), values(), items()

Словарный метод `keys()` возвращает **список ключей** всех элементов словаря.

Следующий программный код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}
```

```
for key in capitals.keys(): # итерируем по списку ['Россия', 'Франция', 'Чехия']
```

```
    print(key)
```

выводит (порядок элементов может отличаться):

Россия

Франция

Чехия

Словарный метод `values()` возвращает **список значений** всех элементов словаря.

Следующий программный код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}
```

```
for value in capitals.values(): # итерируем по списку ['Москва', 'Париж', 'Прага']
```

```
    print(value)
```

выводит (порядок элементов может отличаться):

Москва

Париж

Прага

Словарный метод `items()` возвращает список всех элементов словаря, состоящий из кортежей пар (ключ, значение).

Следующий программный код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}
```

```
for item in capitals.items():
```

```
    print(item)
```

выводит (порядок элементов может отличаться):

```
('Россия', 'Москва')  
( 'Франция', 'Париж')  
  
( 'Чехия', 'Прага')
```

Используя магию распаковки кортежей, можно писать такой код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия':  
'Прага'}  
  
for key, value in capitals.items():  
  
    print(key, '-', value)
```

Такой код выведет (порядок элементов может отличаться):

```
Россия - Москва  
Франция - Париж  
  
Чехия - Прага
```

Распаковка ключей словаря

Если требуется вывести только значение ключей словаря, то мы также можем использовать операцию **распаковки словаря**.

Приведенный ниже код:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия':  
'Прага'}  
  
print(*capitals, sep='\n')
```

выводит (порядок элементов может отличаться):

```
Россия  
Франция  
  
Чехия
```

Начиная с версии Python 3.6 словари являются упорядоченными, то есть сохраняют порядок следования ключей в порядке их внесения в словарь.

Сортировка словаря

Так как словарь состоит из пар, то и отсортировать его можно как по ключам, так и по значениям.

Сортировка по ключам выполняется с использованием функции `sorted()`.

Приведенный ниже код:

```
capitals = {'Россия': 'Москва', 'Англия': 'Лондон', 'Чехия':  
'Прага', 'Бразилия': 'Бразилиа'}  
  
for key in sorted(capitals):  
  
    print(key)
```

будет **гарантированно** выводить ключи словаря в алфавитном порядке, по возрастанию:

```
Англия  
Бразилия  
Россия  
  
Чехия
```

Обратите внимание: функция `sorted()` возвращает отсортированный список ключей, а не словарь. Не путайте встроенную функцию `sorted()` и списочный метод `sort()`. Словари **не содержат** метода `sort()`.

Для сортировки словаря по значениям можно использовать функцию `sorted()` вместе с методом `items()`.

Приведенный ниже код:

```
capitals = {'Россия': 'Москва', 'Англия': 'Лондон', 'Чехия':  
'Прага', 'Бразилия': 'Бразилиа'}  
  
for key, value in sorted(capitals.items(), key = lambda x: x[1]):  
  
    print(value)
```

будет **гарантированно** выводить значения словаря в алфавитном порядке, по возрастанию:

```
Бразилиа  
Лондон  
Москва  
  
Прага
```

Стоит учитывать, что `sorted(capitals.items(), key= lambda x: x[1])` возвращает не словарь, а отсортированный по значению список кортежей.

При сортировке словаря по значениям, мы используем анонимную функцию `lambda x: x[1]`, о ней будет рассказано в следующих уроках.

Примечания

Примечание 1. Как мы уже знаем, с помощью оператора принадлежности `in` можно быстро проверить наличие ключа в словаре. Для проверки наличия значения в словаре можно использовать оператор `in` вместе с методом `values()`, который возвращает список всех значений словаря.

Проверка наличия ключа в словаре:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}

if 'Россия' in capitals:

    print('В словаре есть ключ Россия')
```

Проверка наличия значения в словаре:

```
capitals = {'Россия': 'Москва', 'Франция': 'Париж', 'Чехия': 'Прага'}

if 'Париж' in capitals.values():

    print('В словаре есть значение Париж')
```

Примечание 2. Встроенная функция `sorted()` имеет опциональный параметр `reverse`. Если установить этот параметр в значение `True`, то сортировка будет по убыванию.

Примечание 3. Код для работы со словарями нужно писать таким образом, чтобы он правильно работал при любом порядке обхода с помощью цикла `for`.

Примечание 4. Словарные методы `items()`, `keys()`, `values()` возвращают не совсем обычные списки. Типы этих списков - `dict_items`, `dict_keys`, `dict_values` соответственно, в отличие от обычных списков - `list`. Методы обычных списков недоступны для списков типа `dict_items`, `dict_keys`, `dict_values`. Используйте явное преобразование с помощью функции `list()` для получения доступа к методам списков.